

Integrated Full Stack and Agentic AI Application Development

SE3090 Software Engineering Frameworks | Assignment 1 | Year 3 Semester 1 2026

Group SE_039 | Faculty of Computing | SLIIT

Student ID	Student name	Primary component
IT24103089	Bandara MMSD	A - Context and admissions
IT24103033	Dinuwara K.D.S	B - Planning and triage
IT24102852	Pallawala S R	C - Resource scheduling
IT24102599	Dulshan P A	D - Pharmacy and safety

CareFlow AI connects patient symptom submission, contextual assessment, appointment selection, doctor review and pharmacy operations through a shared ASP.NET Core API and PostgreSQL database.

Submission date: 6 October 2026. This report combines implementation evidence collected on 5 October with subsequent deployment, verification and individual contribution updates. Historical observations are identified separately from final deployment results.

Report Navigation and Submission Access

Section	Contents	Pages
1 to 4	Overview, requirements, architecture and agent workflow	3 to 7
5 to 8	Database, API, client design and security	8 to 12
9 to 12	Testing, agent evaluation, performance and deployment	13 to 18
13	Five architecture decision records	19 to 23
14 to 17	Individual sections A, B, C and D	24 to 44
18 to 19	Group declaration, references and submission access	45 to 47

Deliverable	Access information
Repository	https://github.com/IT24103033/CareFlow-AI_SE_039
React website	https://care-flow-ai-se-039.vercel.app
Backend API	https://careflow-ai-se-039.onrender.com Render-hosted ASP.NET Core API.
Android APK	Download Android APK from Drive Android application distribution folder.
Demonstration video	Open demonstration video folder Recorded system demonstration.
Database evidence	Neon PostgreSQL; schema and development database screenshots are included in this report.
Evaluator accounts	Web login: Doctor - doctor@careflow.ai; Staff - staff@careflow.ai; Admin - admin@careflow.ai. Password for all three: password. Open the React website above and enter the account for the required role. Admin, doctor and staff login were tested on the deployed website. Patient/mobile access: open the app, select Create Account, register with your own test details, then sign in with the credentials you chose.

1 Project Overview and Scope

Business problem

Patient symptom reports, doctor schedules, ward occupancy and pharmacy records are often handled as separate activities. This separation makes it difficult to trace a patient request from initial assessment to an accountable clinical decision and the next operational action. CareFlow AI provides a shared digital workflow for these activities.

Objectives

- Capture symptoms and an optional image from a patient-facing mobile application.
- Use distinct context, planning, action and safety roles to support an auditable triage workflow.
- Require an authenticated doctor to review the assessment and approve consequential appointment actions.
- Provide patient status tracking and staff access to admissions, availability, medicines and prescriptions.
- Persist business data, workflow attempts and review decisions in PostgreSQL.

Boundaries

CareFlow is an academic decision-support prototype. The AI output is a recommendation for human review, not an independently validated clinical diagnosis. The implemented scope is hospital triage, appointments, admissions and pharmacy. It does not include field-nurse dispatch, GPS tracking or home-visit routing; those features belong to the CarePulse reference report and are not CareFlow features.

Component ownership

Bandara owns patient context and admissions; Dinuwara owns triage planning and review; Pallawala owns scheduling and appointment execution; Dulshan owns pharmacy and safety. Integration changes span these boundaries and must be attributed using the actual commit and review history, rather than treating file ownership as exclusive authorship.

2 Requirements and User Roles

Role	Principal responsibilities	Control boundary
Patient	Submit symptoms, select a slot, revise a case, view own appointments and prescriptions.	Patient identity must come from the authenticated session.
Doctor	Review assessments, approve or reject, request revision, issue prescriptions.	Active doctor identity and valid workflow state.
Staff	Patient records, admissions, ward and pharmacy operations.	Restricted operational actions; no doctor approval impersonation.
Admin	Account administration, patient management and overview.	Administrative privileges do not imply clinical decision authority.

Functional requirements

The system supports identity, patient records, ward admission, symptom assessment, review history, available-slot discovery, appointment approval, medication inventory and prescription dispensing. Collection APIs and screens provide differing levels of search and filtering; the triage review queue includes server-side pagination and urgency sorting.

Nonfunctional requirements

Both clients use the same public API and persistence model. External calls need bounded execution, safe failures and secret protection. Repeated requests must not duplicate consequential actions. UI states must distinguish pending work, successful assessment, failed assessment and failed downstream execution. Audit summaries should retain structured results without hidden reasoning or credentials.

Acceptance workflow

A non-emergency patient request starts in Flutter, is persisted and assessed through A and B, receives D safety validation, and exposes suitable slots. The patient selects a slot through C. A doctor reviews the case in React; the appointment is approved and confirmed only through the authorized flow. The patient then sees the updated outcome. This manual-selection design supersedes the earlier automatic-booking prototype.

3 Full Stack Architecture

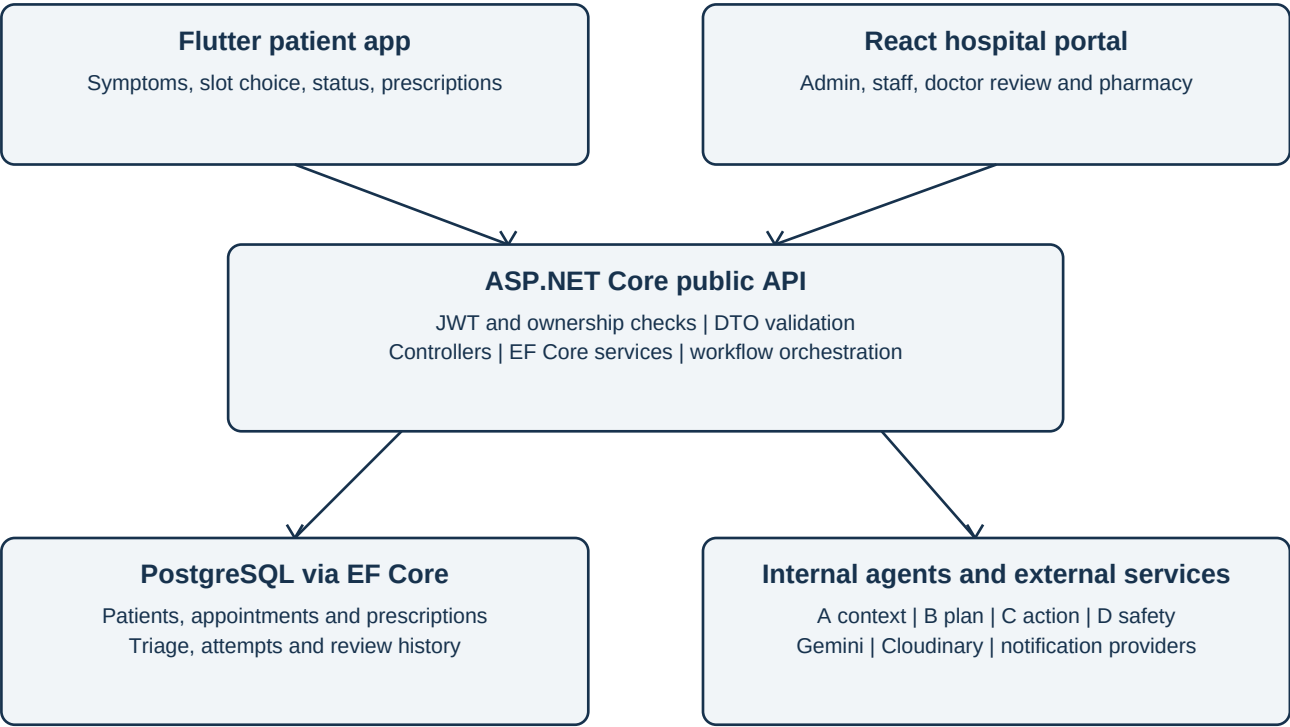


Figure 1. Logical deployment and integration architecture derived from the repository.

React and Flutter communicate with ASP.NET Core. Controllers apply endpoint rules and DTO validation; services contain planning, scheduling, pharmacy and notification logic. EF Core maps the application model to PostgreSQL. The orchestrator is a referenced .NET project hosted with the API, rather than a separate public service.

Gemini supplies model-based context analysis and assessment. Cloudinary stores uploaded images. NotificationService uses configured HTTP providers for email and SMS. Provider availability and delivery are separate from successful database persistence.

Repository area	Responsibility
backend-api	Public API, EF model, migrations, services, hosted workflow recovery
ai-orchestrator	Agent contracts, agent implementations and controlled appointment tools
web-admin	React role-specific portal
mobile_app	Flutter patient application and device features
backend-api.Tests and client tests	Automated verification
.github/workflows	Build, test and container publishing automation

4 Agent Roles and Contracts

Role	Input and output	Controlled capability
A DomainAnalysisAgent	Symptoms and patient reference to risk, flagged factors and ward recommendation.	Patient-history access and Gemini analysis.
B PlanningAgentService	Symptoms, profile snapshot and A result to a versioned clinical plan and execution summary.	Context retrieval, assessment client and deterministic validation.
C AppointmentActionAgent	Doctor, patient, date and times to a typed AppointmentActionResult.	Find slots, check conflicts and create tentative booking.
D PharmacyAiService	Triage symptoms/plan or prescription items and patient context to safety findings.	Emergency rules and medication safety checks.

A and B use model-backed analysis. C performs a controlled tool workflow and D applies deterministic domain rules. Distinct roles are evidenced by different inputs, responsibilities and tool permissions, not by requiring four separate models.

Planning and state

ClinicalPlan contains SchemaVersion, Objective, SuggestedSpecialist, UrgencyLevel, RecommendedAction, Rationale, Steps, Warnings and Execution. Steps identify the responsible role, allowed tool, dependencies and approval requirement. Planning events retain operation, outcome and duration. AgentWorkflows records the triage ID, agent, status, input/output payloads, errors and timestamps.

Visible execution

The downstream helper records safety output and available slots as AppointmentAgent ActionRequired. The patient-facing book-slot route invokes C to validate and reserve the selected slot. Proposed steps and executed outcomes must be interpreted separately. The existence of a plan is not evidence that each downstream action completed.

4 Agent Workflow and Safe Failure

Stage	Expected state transition	Evidence to inspect
Submit	New triage and running planning attempt	TriageRecords and AgentWorkflows
Assess	Validated plan to InReview; failed plan to AssessmentFailed	Schema validation and execution events
Select slot	ActionRequired to tentative reservation	AvailableSlots and linked appointment ID
Review	Pending to Approved, Rejected or RevisionRequested	Authenticated doctor, version token and review history
Revise	Patient update triggers a new assessment attempt	Old review preserved and new attempt recorded
Complete	Booking confirmation or explicit downstream failure	Current appointment row and final UI state

Human approval

Doctor review checks an active doctor profile, current triage status and ExpectedUpdatedAt. The case decision and linked appointment action are performed within a transaction where supported.

Appointment records also retain approval status, approving doctor and approval time. A transaction alone does not establish that all concurrency edge cases have been tested.

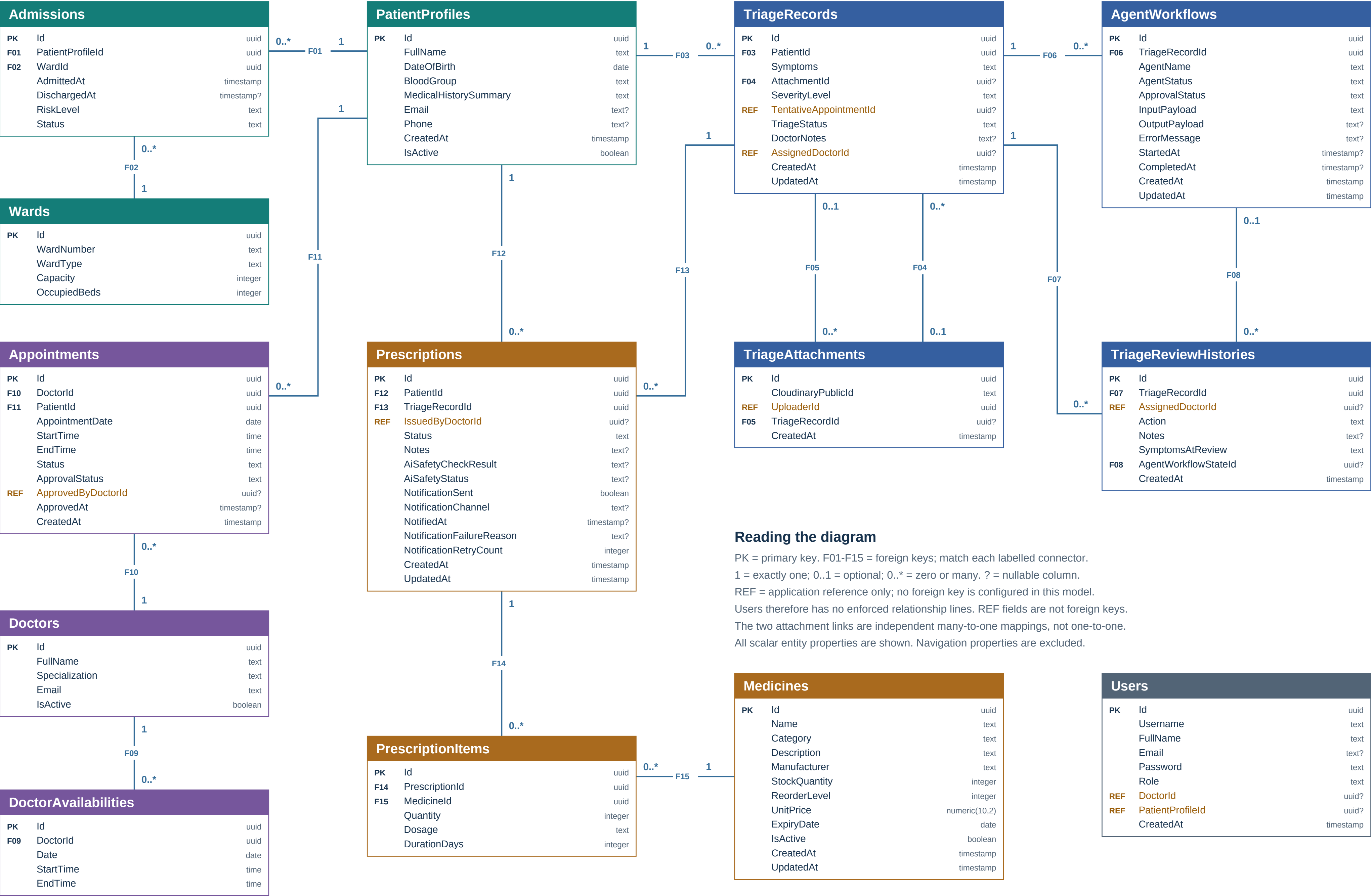
Known verification boundaries

An early emergency path can return a rule-generated critical recommendation and bypass the normal four-role chain. It must be demonstrated as an emergency path, not as evidence that all four agents executed. The inspected context integration still passes PatientId to a name-based A history lookup. Recovery currently searches for a recent tentative patient booking rather than using a unique workflow key. These limitations require correction or explicit disclosure in evaluation.

Appointment notification routing is recorded as unsupported in parts of the triage flow. Prescription notification functionality is separate and must not be used as evidence that appointment messages were delivered.

5 Database Design | Complete Entity Relationship Diagram

14 mapped tables | 15 foreign-key relationships | Current EF Core model and migration snapshot | 5 October 2026



Source: backend-api/Models/*.cs; Data/ApplicationDbContext.cs; Migrations/ApplicationDbContextModelSnapshot.cs.

Code-derived schema; live database migration state not verified. A = teal | B = blue | C = purple | D = amber | Authentication = grey.

5 Schema Integrity and Migration Strategy

Component	Principal entities	Integrity concerns
A	Users, PatientProfiles, Wards, Admissions	Identity association, active records and bed capacity.
B	TriageRecords, AgentWorkflows, TriageReviewHistories, TriageAttachments	Case ownership, attempt linkage, version checks and attachment access.
C	Doctors, DoctorAvailabilities, Appointments	Time ranges, overlap prevention, approval identity and status.
D	Medicines, Prescriptions, PrescriptionItems	Patient/case consistency, dosage checks and stock integrity.

EF Core migrations and the model snapshot are included in the repository. Seed records support demonstrations, but seeded dates and accounts must not be confused with production data. Several entities use UpdatedAt as an optimistic concurrency token; this is not a claim that all entities use PostgreSQL xmin or that every conflict is translated correctly.

The CI workflow provisions PostgreSQL and creates a schema with EnsureCreatedAsync. That verifies schema creation through the current model, not replay of the entire migration chain. A clean database migration test remains a separate requirement.

6 API Design and Component Endpoints

Component	Representative endpoints	Business operation
A	POST /api/auth/register-patient; GET/POST /api/PatientProfiles; GET /api/PatientProfiles/search; POST /api/Admissions/allocate-ward; POST /api/Admissions/analyze-risk	Identity and context-aware admission support.
B	POST /api/triage; GET /api/triage; GET /api/triage/review-queue; PATCH /api/triage/{id}/review; POST .../{id}/revise; POST .../{id}/retry; POST .../{id}/book-slot	Assessment, revision, approval and delegated execution.
C	GET /api/DoctorAvailability/slots; POST /api/Appointments/tentative; POST .../{id}/approve; POST .../{id}/confirm; POST .../{id}/cancel	Conflict-checked reservation and human-approved confirmation.
D	GET/POST /api/Medicines; PATCH .../{id}/restock; POST /api/Prescriptions; PATCH .../{id}/approve; PATCH .../{id}/dispense; PATCH .../{id}/notify-retry	Safety-checked prescribing, stock updates and notification retry.

The controllers use asynchronous operations and DTOs for workflow requests. Outcomes include created, validation failure, unauthorized, forbidden, not found and conflict responses. JWT bearer authentication supplies role, patient and doctor claims. Swagger documents the API in development; production Swagger availability is not established by the deployment evidence.

Every component has at least four identifiable API operations across its owned controllers. Endpoint count alone is not evidence of complete security, error handling or user-facing integration.

7 React and Flutter Design

Client and role	Implemented screens	State and integration
React Admin	Patient/ward counts, patient management, staff registration	AuthContext, hooks, React Router and apiFetch.
React Staff	Patient records, wards, history search, risk analysis, availability and pharmacy	Shared backend; role-dependent navigation.
React Doctor	Triage queue, review/revision, assessment details and prescriptions	Versioned decisions and refresh after actions.
Flutter Patient	Registration/login, symptoms/image upload, case history/revision, appointments, prescriptions and ward status	Secure token storage, service classes and local widget state.

The web portal supports hospital operations while Flutter supports the patient journey. The mobile image picker and upload flow provide a meaningful device capability. Prescription QR display and local notifications are additional features; local notification display is not proof of external SMS or email delivery.

Workflow presentation

Web and mobile triage views show recommendation, severity, doctor notes, safety findings, scheduling outcome and notification outcome when returned by the API. The latest manual-slot design adds available-slot selection before booking. Current appointment rows should be used for live status; stored agent outputs are historical evidence.

8 Security and Limitations

Implemented controls

The backend issues JWTs, uses hashed passwords, restricts many endpoints by role and checks patient/doctor claims for sensitive workflows. Flutter uses secure storage for session material. Triage review checks versions and allowed transitions. Appointment tools now use service/provider interfaces rather than the earlier hardcoded self-HTTP approach.

Historical security review and limitations

- The inspected PatientProfiles update/delete actions lacked explicit authorization attributes; final-release access controls were not independently re-audited.
- Earlier inspection identified credential-handling concerns, including secret logging and hardcoded fallbacks; a separate final security review is outside the recorded test results.
- Earlier context integration used partial-name history lookup while B supplied a patient ID, creating an identity-matching risk.
- The inspected recent-booking recovery heuristic presents a case-linkage risk without a workflow-specific idempotency key.
- Manual book-slot should enforce case state, repeat-request behavior and consistent response mapping. The inspected implementation passes the appointment workflow to a mapper otherwise used for planning output.
- Provider payloads and error messages must be minimized before exposing them to patients.

These findings were recorded during the earlier code inspection and are not a fresh audit of the deployed release. CareFlow AI is an academic prototype; no HIPAA or ISO certification is claimed.

9 Software Testing Report

Check	Observed result	Interpretation
Backend dotnet test	101 passed, 1 skipped, 102 total	Current application and test projects compiled; suite completed.
React node tests	10 passed, 0 failed	Triage API contract and failure-handling checks.
React production build	Passed	Vite built the current web application.
Flutter static analysis	Passed: no issues found	flutter analyze --no-pub passed after the later deployment configuration changes.
PostgreSQL concurrency	Not established by this run	Appointment concurrency test is skipped for in-memory provider.
Deployment and APK	Render and Vercel live; APK linked	Role logins tested by the team; installed-device test evidence is not recorded here.

The latest supplied backend run on 6 October 2026 reported 101 passed, 1 skipped and 0 failed in approximately four seconds. React tests and production build passed during deployment preparation. Flutter static analysis subsequently passed. Test duration is not API response latency.

Coverage

Backend suites include authentication/authorization integration, admissions, configuration, planning, domain analysis, triage review, pharmacy safety, scheduling and orchestration. Flutter tests exercise validation, submission states, timeout messaging, revision/history behavior and late asynchronous responses. React tests validate request contracts, status handling and assessment/plan parsing.

Coverage limitations

Most backend data tests use EF InMemory. Ignoring transaction warnings does not validate rollback or PostgreSQL locking. The current React tests are not comprehensive rendered-component or browser tests. Static analysis does not replace an installed-device end-to-end test.

9 End to End Test Record

Case	Procedure and expected result	Evidence status
E2E 01 Normal case	Submit in Flutter, inspect A/B/D results, select C slot, approve in React, refresh patient booking.	Earlier booking evidence; team-reported end-to-end testing.
E2E 02 Revision	Doctor requests clarification; patient revises; new attempt appears while old review remains.	Selected automated coverage; no final cross-client trace recorded.
E2E 03 Rejection	Reject with notes; release linked tentative slot; patient sees rejection.	No separately recorded final-release result.
E2E 04 Emergency	Emergency rule produces escalation; no routine appointment is created.	Earlier saved case was approved through the emergency shortcut; not a four-agent golden case.
E2E 05 Failure	Provider unavailable or invalid result; no invented urgency; explicit follow-up/retry state.	Selected unit/widget tests; no deployed failure trace recorded.
E2E 06 Isolation	Patient A cannot read or change patient B's case or appointment.	Endpoint tests; separate manual isolation result not recorded.

Historical database inspection on 4 October recorded a non-emergency case with completed context analysis, validated planning, safety validation and a real tentative appointment for the same patient. It also showed missing context history in A. This is evidence for an earlier build, not proof that the current release satisfies all acceptance criteria.

10 Agentic AI Evaluation Report

Evaluation method

Evaluate the workflow with fixed synthetic inputs, deterministic schema/business assertions and human review. A successful model response is insufficient: the correct tools, identity, approval gate and durable outcomes must also be verified. No LLM-as-judge score is claimed.

Golden or adversarial case	Required assertion	Current evidence
Routine valid symptoms	Valid plan; supported specialty; explicit slot outcome; no confirmation before approval.	Planning tests and historical workflow evidence.
Invalid structured output	Reject malformed or unsupported output; record failure.	Planning/domain validation tests.
Prompt injection in symptoms	Input cannot change roles, tool permissions or approval requirements.	No dedicated evaluation result recorded.
Provider timeout	Bounded failure; no fabricated recommendation; safe retry.	Selected tests; no deployed failure result recorded.
Conflicting appointment	Only permissible reservation succeeds; return a structured conflict.	Service tests; concurrent database proof incomplete.
Wrong identity	Reject cross-patient access and spoofed reviewer.	Authentication integration test coverage.
Emergency	Escalate; skip routine scheduling; label rule-based processing accurately.	Deterministic rules and prior record.

11 Performance Report

Measured observations

A historical single workflow recorded context retrieval at 301 ms, domain analysis at 2530 ms and symptom assessment at 2178 ms, with one model attempt. These sum to approximately 5.0 seconds for those recorded operations only. Booking, upload, network transit and client rendering are not included. This is one observation from an earlier build, not a benchmark or percentile.

Measurement limitations

The available performance evidence is a historical single-workflow trace. Concurrent-load results, p50/p95 latency, throughput and provider failure rates were not measured in the evidence recorded here.

Workload	Record	Evaluation protocol
Read-only API	Request count, concurrency, p50/p95, failures	Synthetic dataset; 1, 5 and 10 concurrent users.
Triage submission	A/B latency, total workflow time, model attempts	Small bounded synthetic batch; separate model time from DB time.
Booking race	Accepted/rejected requests and final row count	Two independent clients target the same slot.
Review race	One accepted decision; explicit loser outcome	Two authenticated doctors submit the same version.
Database	Query time and integrity after workload	Disposable PostgreSQL test database.

12 Deployment and Reproducibility

The current repository contains a multi-stage .NET 8 Dockerfile. It publishes both backend-api and ai-orchestrator into one runtime container listening on port 8080. React is a Vite static build. Flutter must be built as a signed Android APK using the final HTTPS API endpoint.

Area	Configuration or command	Status
API	Docker build from repository root; PostgreSQL connection; JWT and Gemini settings.	Render deployment succeeded; API service live.
React	npm ci; npm run build; publish dist; VITE_API_BASE_URL.	Deployed to Vercel; role logins tested.
Flutter	flutter pub get; flutter build apk --release after endpoint/signing configuration.	API configuration updated; APK distribution link included.
Database	Reviewed EF migrations; restricted PostgreSQL credentials.	Neon used in earlier read-only verification.
Providers	Cloudinary and configured email/SMS endpoints.	External email/SMS delivery not established.

Environment variable names

ConnectionStrings__DefaultConnection; Gemini__ApiKey; Gemini__Model; Jwt__Secret; Jwt__Issuer; Jwt__Audience; CLOUDINARY_URL; Notifications__EmailEndpoint; Notifications__SmsEndpoint; Notifications__ApiKey. Do not include secret values in the report, APK, web bundle or repository.

Local startup order

Restore and configure the API, apply reviewed migrations to the intended database, start the backend using its project path, then start React and Flutter. From the repository root: dotnet run --project backend-api --launch-profile http. From web-admin: npm install followed by npm run dev. From mobile_app: flutter pub get followed by flutter run. Client URLs must match the chosen host; emulator loopback addresses do not work on physical phones.

12 Deployment Acceptance and Demonstration

Deployment record

The backend deployment completed successfully on Render and the React website is live on Vercel. The team tested admin, doctor and staff login. Mobile services use the deployed HTTPS API, and the APK and demonstration recording are available through the links on page 2.

Ten minute demonstration plan

Time	Demonstration
0:00 to 1:00	Overview, architecture and four roles.
1:00 to 3:00	Patient login, symptoms and image upload; show assessment and slot selection.
3:00 to 5:00	Inspect agent outputs and doctor approval in React.
5:00 to 6:00	Refresh patient appointment outcome; demonstrate revision or failure branch.
6:00 to 7:30	Prescription safety, approval, dispensing and notification outcome.
7:30 to 9:00	Database state, tests, CI and contribution evidence.
9:00 to 10:00	Deployed URLs, APK and limitations.

Use synthetic accounts and records. The submitted application may run its own agents during the viva, but external assistants must not answer viva questions or modify the work. Each member should be ready to explain one controller, schema relationship, UI state transition, agent contract and test from their component.

13 ADR 001 React State Management

Context

Session data is shared across protected routes, while filters, forms and review actions are local to individual screens.

Alternatives

React Context with hooks; Redux Toolkit; a dedicated server-state library.

Decision

The inspected implementation uses AuthContext for identity and React hooks for screen state. Retain that lightweight approach for the current scope.

Consequences

Limited boilerplate and clear role routing are benefits. Manual refresh, duplicate fetch logic and race handling need explicit tests. Add a query-cache library if shared server state becomes substantially more complex.

Evidence

web-admin/src/context/AuthContext.jsx; src/App.jsx; src/pages/TriageReview.jsx

13 ADR 002 Flutter State Management

Context

The patient application needs secure session storage and asynchronous forms, history and booking screens.

Alternatives

Local StatefulWidget state with services; Provider or Riverpod; BLoC.

Decision

The inspected screens primarily use local state and service/repository boundaries, with flutter_secure_storage for session persistence. A provider dependency exists, but that alone does not prove consistent Provider adoption.

Consequences

This keeps screen logic understandable and supports repository-based widget tests. Cancellation, mounted checks and shared refresh behavior require discipline. A central session and workflow store would reduce duplicated state as the app grows.

Evidence

mobile_app/lib/services/auth_service.dart; services/triage_service.dart;
screens/triage_dashboard_screen.dart

13 ADR 003 Agent Orchestration

Context

The assessed workflow must combine model output, controlled scheduling actions, deterministic safety and human review within the mandatory ASP.NET backend.

Alternatives

A custom .NET orchestrator; a separate Python agent service; a general-purpose agent framework.

Decision

CareFlow uses custom C# orchestration, DI contracts, a hosted recovery worker and a referenced orchestrator project.

Consequences

One runtime simplifies deployment and shared identity. The group owns transition rules, timeouts, retries and audit correctness. Strongly typed contracts and integration tests are essential; a named agent class alone does not prove safe orchestration.

Evidence

backend-api/Services/PlanningAgentService.cs; TriageOrchestrationHelper.cs; WorkflowManager.cs;
ai-orchestrator/Agents

13 ADR 004 Workflow Persistence

Context

Model outputs vary, while business entities and doctor decisions require durable relational integrity and auditability.

Alternatives

Relational business tables plus versioned serialized summaries; an event store; transient memory only.

Decision

Use PostgreSQL entities for cases, appointments and reviews, with versioned serialized input/output summaries in AgentWorkflows.

Consequences

Business identities remain queryable and review history is retained. JSON compatibility and duplicate status representations require careful mapping. Current appointment status should come from its live entity, not an old serialized booking result.

Evidence

backend-api/Models/AgentWorkflowState.cs; TriageRecord.cs; TriageReviewHistory.cs;
Data/ApplicationDbContext.cs

13 ADR 005 Deployment Platform

Context

The group needs a reproducible API, web portal, shared PostgreSQL database and installable APK with a short delivery window.

Alternatives

Render Docker plus static hosting; Azure managed services; a self-managed VM.

Decision

The API is deployed as a Docker web service on Render, and the React/Vite application is deployed on Vercel. Neon provides PostgreSQL storage and Cloudinary supports uploaded images. The Android APK is distributed through the linked Drive folder.

Consequences

Managed hosting reduces server setup. Free-service sleep, provider quotas, cold starts and external dependencies may affect demonstrations. Secrets, migrations, health checks, HTTPS client configuration and final link verification remain release responsibilities.

Evidence

Dockerfile; web-admin/vercel.json; mobile_app/lib/services/api_config.dart; deployed service URLs on page 2

14 Individual Report Component A

Bandara MMSD | IT24103089 | Patient admissions, wards and shared user management

Contribution statement

My primary responsibility was Component A across ASP.NET Core, PostgreSQL, React, Flutter and domain analysis. I designed PatientProfiles, Wards and Admissions with EF Core, developed patient/admission APIs, and contributed to the transition from local PostgreSQL to the shared Neon database.

Database and backend

The admission workflow links a patient to a ward and updates OccupiedBeds in the same transaction. The current route is POST /api/Admissions/allocate-ward. It checks ward existence and capacity before saving. A transaction supports consistent writes; concurrent capacity enforcement still requires separate verification.

React, Flutter and shared authentication

I built staff-facing patient tiles, patient history search through GET /api/PatientProfiles/search, the ward-capacity display and the Admit to Ward modal. Mobile work enabled patients to view medical history and blood group through patient-profile API integration. I also contributed the role-based web login, backend user models and role verification. Mobile authentication and extended account management were shared work completed by a teammate.

Domain analysis agent

The Gemini-based domain agent combines symptoms with patient history and requests structured JSON containing risk and a ward recommendation. My work included provider failure handling and the migration from a local Ollama integration. The supplied report describes a high-risk fallback; fallback routing must be distinguished from a successful model assessment and reviewed by authorized staff.

DevOps, evidence and challenges

I contributed the backend GitHub Actions pipeline and investigated build failures caused by DTO changes, including AdmissionRequestDto. The supplied screenshots show project setup PR #1 merged and admissions PR #7 closed. These statuses are preserved as historical evidence rather than described as current passing CI.

Challenges included aligning the React admission interaction with backend state changes, enforcing JSON response structure, configuring cloud database access and resolving schema/merge mismatches. The screenshots that follow document the supplied development evidence; current deployment and final test results are recorded separately.

14 Individual AI Usage Log

Bandara MMSD | IT24103089

Tool	Prompt or task	Student modification
Gemini	"Generate ASP.NET Core C# code to connect to a Neon PostgreSQL database and build the PatientProfiles and Wards tables using EF Core."	Adapted the generated connection strings and DbContext classes to fit the project's repository structure.
Gemini	"How to swap a local Ollama LLM integration for the Gemini Cloud API in C# while enforcing a strict JSON output schema?"	Integrated the provided HttpClient logic and responseType = "application/json" into the DomainAnalysisAgent.
Antigravity IDE	"Analyze attached UI designs and generate a React login page and Hospital Staff landing page with a patient search bar and admission modal."	Reviewed the generated UI components, manually wired the fetch() API calls to the C# backend, and adjusted state management for the modal triggers.
Gemini	"Explain why my GitHub Actions CI pipeline failed with a build-and-test error after modifying the AllocateWard method."	Used the diagnostic advice to run dotnet build locally, identifying uncommitted test files that were missing the new AdmissionRequestDto schema.

Sanuthmi reports reviewing generated models and UI, wiring API calls, adjusting modal state and using local dotnet build to diagnose the DTO mismatch. These statements do not replace dated session records.

14 Individual Reflection and Declaration

Bandara MMSD | IT24103089

Individual AI reflection

The implementation of Generative AI technology has been very instrumental in speeding up the lifecycle of development of Component A, especially when creating the boilerplate architecture and creating the React code from the visual interface ideas. From the onset of development of the component, using AI to quickly create Entity Framework Core models and REST API controllers laid a good ground for concentrating on the relational logic such as Admissions.

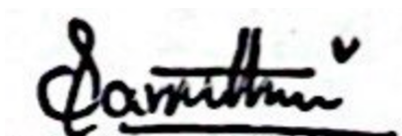
The major learning point here was the process of moving from the local LLM (Ollama) to the cloud-based Gemini API to work with the Domain Analysis Agent. Although the initial set-up entailed dealing with the limitations of the network connection and API keys within the appsettings.Development.json, the cloud API turned out to be more dependable in terms of ensuring that the JSON Schema was enforced. This made sure that the AI risk assessment could be deserialized without crashing the C# back-end because of any formatting hallucination. The necessity of the safety net through the fallback error handling (like falling back to "High Risk" during a 503 error) was proved.

Working with Antigravity IDE to create the React front end from the mocks revealed how powerful but limited the capabilities of AI code writing are. On the one hand, it perfectly implemented the CSS styling and routing of the components for the Hospital Staff dashboard and the role-based login, while on the other hand, it was not intelligent enough to be able to automatically connect the separate PatientProfiles and Wards pages. I personally had to design and prompt AI to implement the "Admit to Ward" flow to actually perform the business logic. In essence, AI is just an advanced syntax generator and debugging assistant, but enforcing data integrity, resolving Git merge conflicts, and maintaining the overarching system architecture remain strictly as job of humans.

Individual declaration

I, Bandara MMSD, declare that the work presented in this report is my own original work, except where acknowledged through the AI Usage Log and standard academic references. I confirm that all AI-generated code has been reviewed, tested, and modified to meet the project's specific architectural requirements.

Date: 06/10/2026



14 Component A Evidence Project Setup

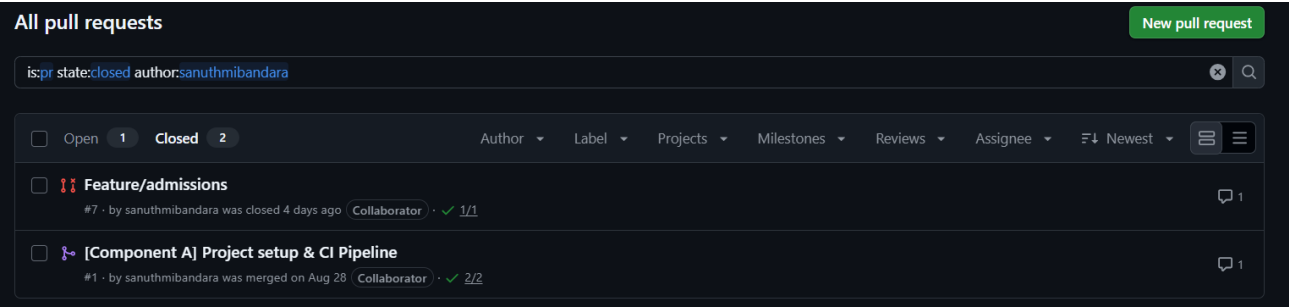


Figure A1. Supplied pull-request list for Component A, showing admissions and project setup work.

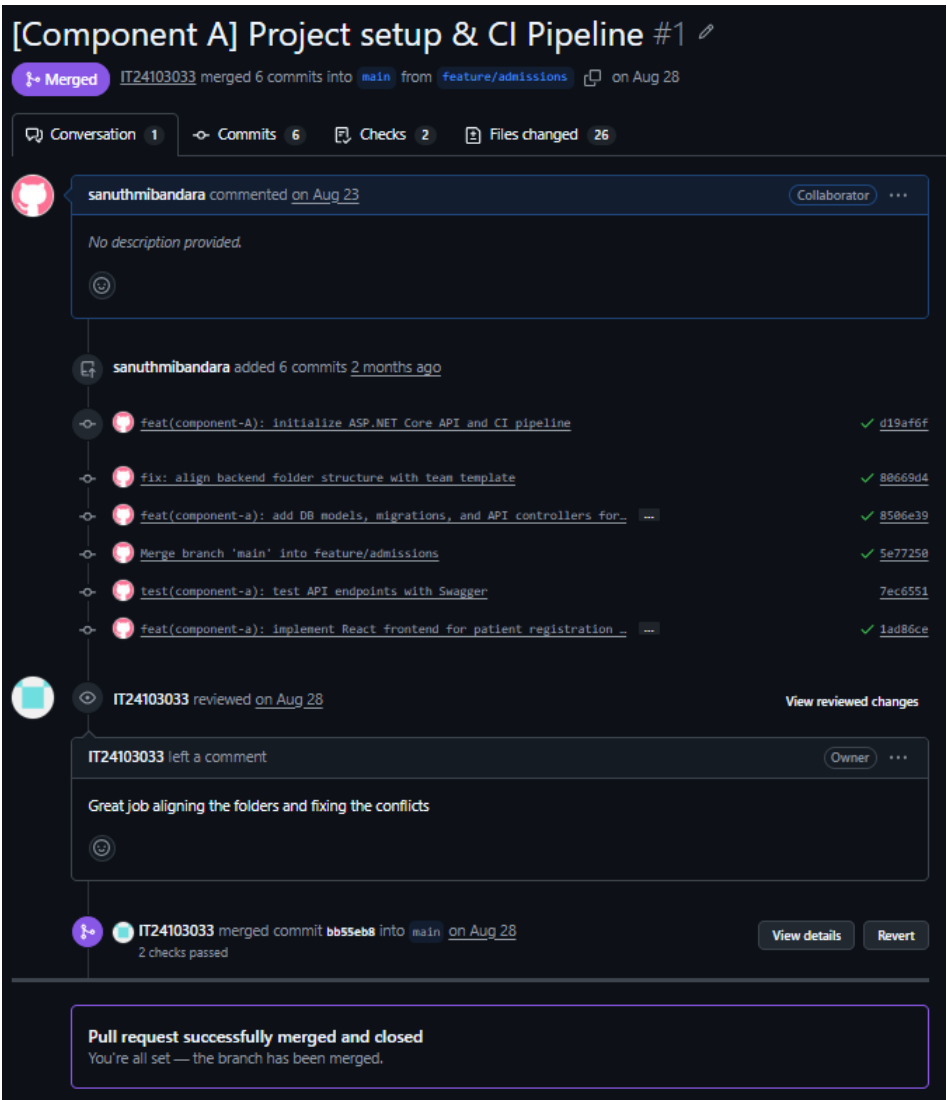


Figure A2. Project setup and CI pipeline PR #1 shown as merged. Historical development evidence.

14 Component A Evidence Admissions Review

Feature/admissions #7

Closed

sanuthmibandara wants to merge 4 commits into main from feature/admissions

Conversation 1

Commits 4

Checks 1

Files changed 40

sanuthmibandara commented 2 weeks ago

Collaborator

Component A: Patient Admissions & Ward Management

sanuthmibandara added 4 commits last month

feat(component-a): flutter mobile application setup, db connection, b...

22fcce1

feat(component-a): complete Gemini AI integrations

2d75cb7

feat(component-a): build patient history search interface

372a0e9

feat(component-a): implement role-based UI, staff dashboard, and admi...

30dc25d

IT24103033 commented 4 days ago

Owner

Closing this PR — the changes from this branch were taken, conflicts resolved, and merged into main directly via a separate branch.

IT24103033 closed this 4 days ago

Figure A3. Admissions PR #7 shown as closed. The discussion describes continued work through another branch; closed must not be interpreted as directly merged.

14 Component A Evidence Database

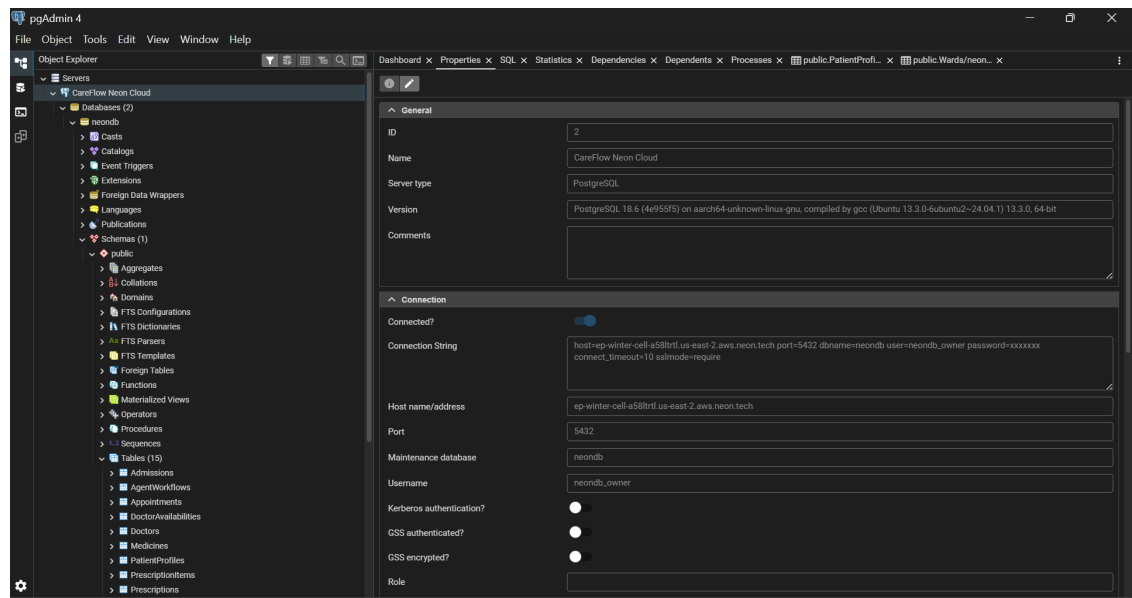


Figure A4. Supplied pgAdmin cloud connection view; source password is masked. Connection visibility alone does not prove all cloud workflows.

The screenshot shows the 'PatientProfiles' table view in pgAdmin 4. The table has 15 rows and 6 columns: 'Id [PK] uuid', 'FullName text', 'DateOfBirth date', 'BloodGroup text', 'MedicalHistorySummary text', and 'Registered via Mobile App'. The data is as follows:

Id [PK] uuid	FullName text	DateOfBirth date	BloodGroup text	MedicalHistorySummary text	Registered via Mobile App
1aceff5e-44c0-4fe7-bb9d-eccc634c1a...	saman	2000-01-06	O+	Registered via Mobile App	
339a3bc6-f222-4d2b-9037-56f8bfa8d...	Kamala Pereraaaa	1981-02-09	B-	Allergy to seafood	
4e3edc4a-0be5-4102-a9b1-73be9c655...	Test Patient	1990-01-01	O+	None	
52629963-7e46-49fe-badf-8a8b7af6d...	Dammika Silva	2026-12-02	O-	No prior conditions	
55555555-5555-5555-5555-55555555...	Sarah Roahel	1985-03-12	O+	No known allergies. Previous appendectomy.	
55555555-5555-5555-5555-55555555...	Manik Perera	1972-11-05	A-	Type 2 Diabetes, Hypertension.	
55555555-5555-5555-5555-55555555...	Amanda Samarasing...	1990-07-22	B+	Asthma, treated with inhalers.	
55555555-5555-5555-5555-55555555...	Daham Weerasinghe	1950-01-30	O-	Coronary artery disease, pacemaker fitted 2018.	
95502301-b2e8-4d06-afe1-97a1e48cd...	Kamal Perera	1995-05-15	O+	No known allergies	
ad39d6c3-94ea-47b9-818e-4c34906b...	Sujana Dinuwara	2004-10-17	AB+	Registered via Mobile App	
d6a88a71-218d-4e48-86f6-ba5eb041a...	Kapila Edirisinghe	1990-01-01	O+	None	
d6f04023-c8ac-437e-b9af-4bd512b2a...	Chandani Silva	1990-01-01	O+	None	
d9aafc50-dce4-4548-9961-99430b3bd...	Jane Doe	2000-01-01	O+	Registered via Mobile App	
e76819d0-57ce-46a2-9519-bd3e4727f...	Jane Doe	2000-01-17	O+	Registered via Mobile App	
fe2b25f5-2354-46dd-9c4f-e74c240da4...	Nethra Silva	1996-10-09	AB+	Has had a severe stomach pain for the past 5 weeks continuously. Eating seafood makes the condition worse	

Figure A5. Supplied PatientProfiles table view showing profile and history fields.

The screenshot shows the 'Wards' table view in pgAdmin 4. The table has 5 rows and 5 columns: 'Id [PK] uuid', 'WardNumber text', 'WardType text', 'Capacity integer', and 'OccupiedBeds integer'. The data is as follows:

Id [PK] uuid	WardNumber text	WardType text	Capacity integer	OccupiedBeds integer
44444444-4444-4444-4444-44444444...	G-01	General	50	45
44444444-4444-4444-4444-44444444...	I-01	ICU	15	13
44444444-4444-4444-4444-44444444...	M-01	Maternity	30	20
44444444-4444-4444-4444-44444444...	P-01	Pediatrics	25	10
510f4c44-ebfe-46f6-a8d8-1c106e734...	W-101	General	20	5

Figure A6. Supplied Wards table view showing capacity and occupied-bed values.

14 Component A Evidence Staff Interface

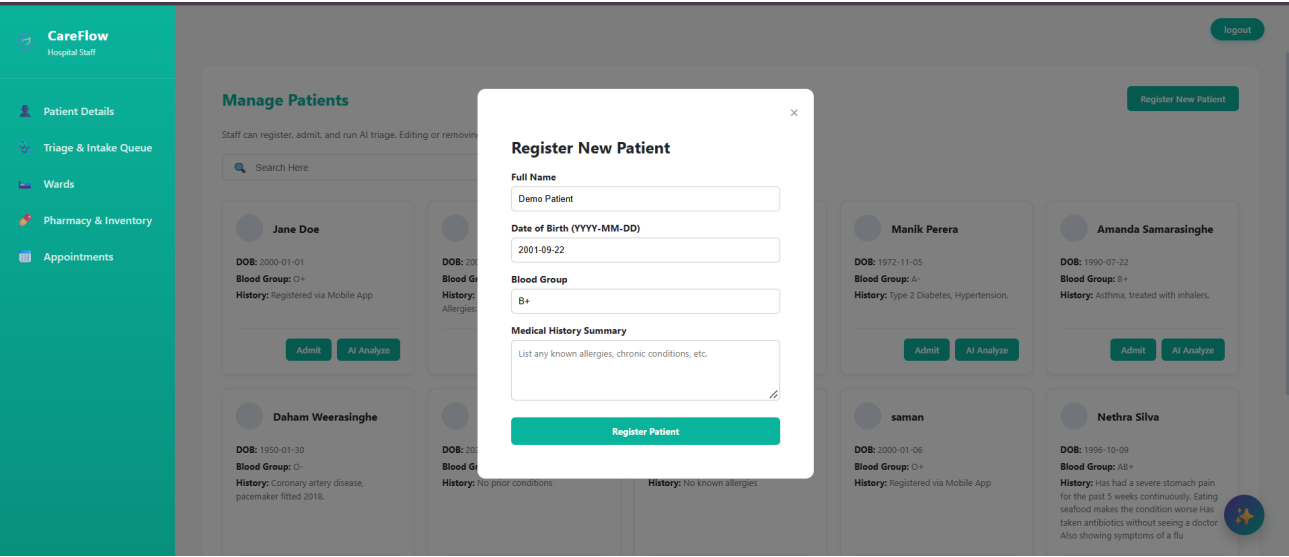


Figure A7. Patient registration modal in the staff interface.

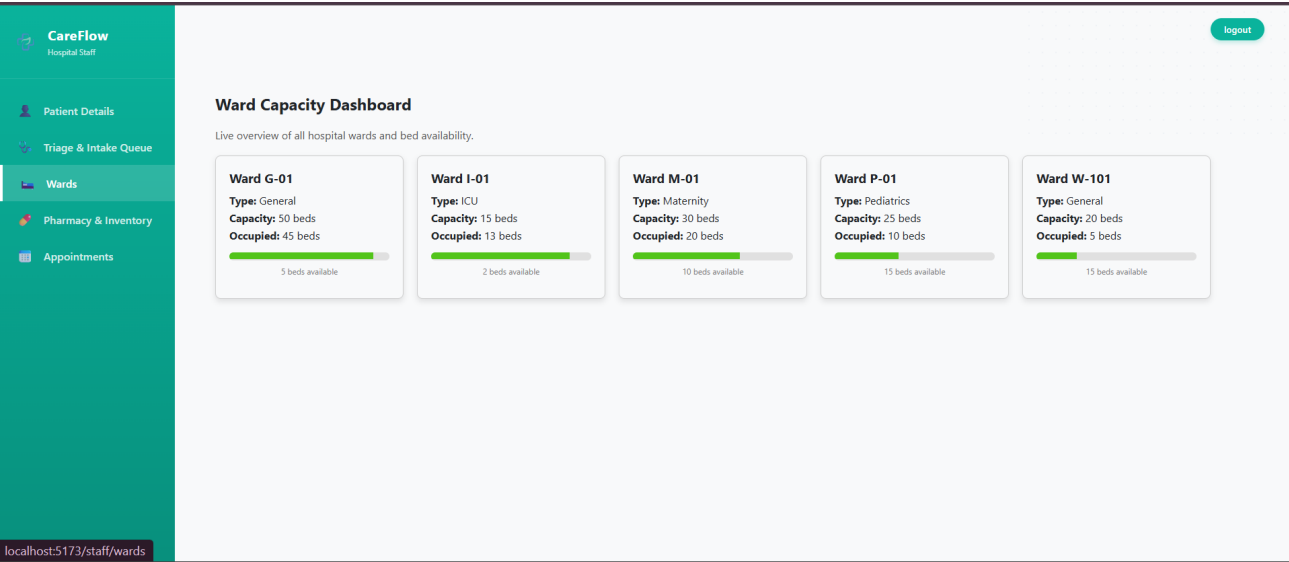


Figure A8. Ward-capacity dashboard with occupancy indicators.

14 Component A Evidence Analysis and Mobile

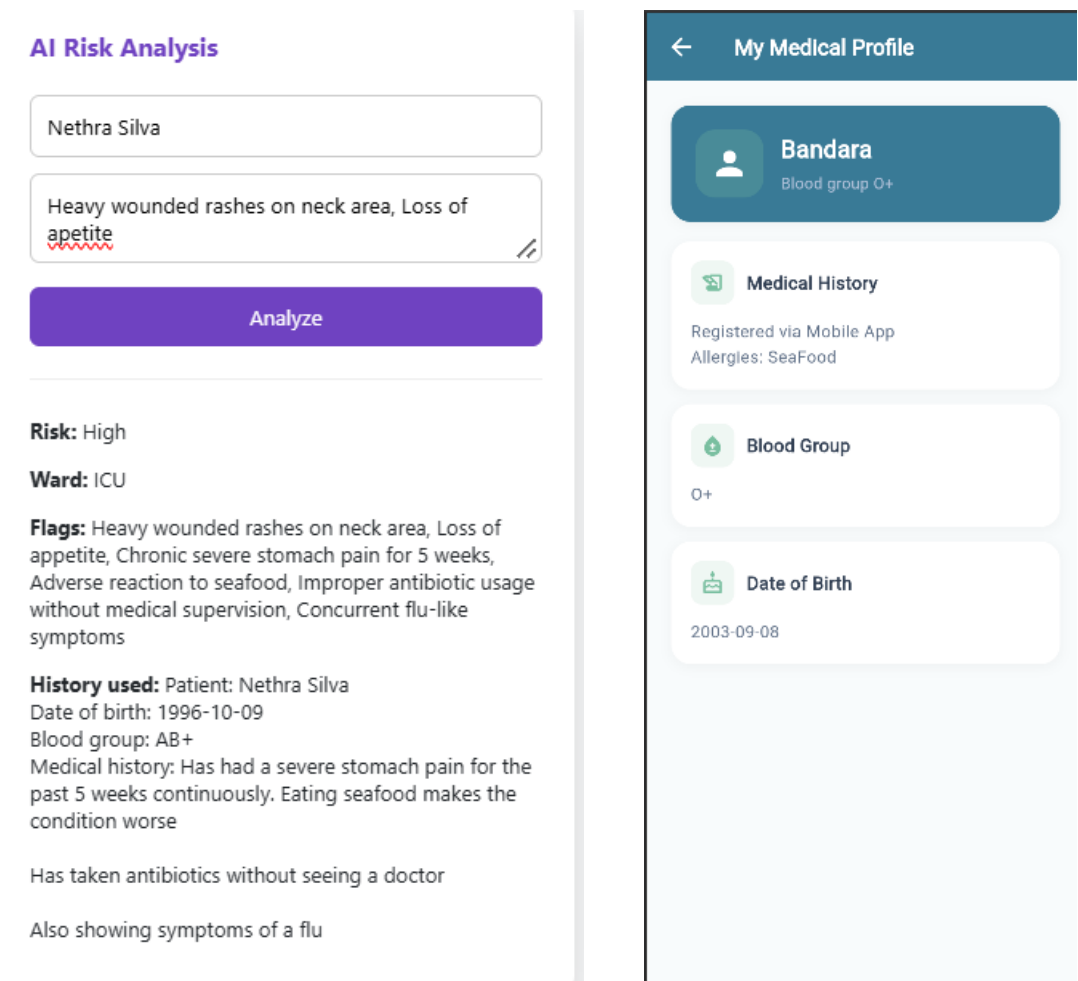


Figure A9 (left). Supplied risk-analysis result with risk, ward recommendation, factors and patient history. Figure A10 (right). Flutter medical-profile screen displaying history, blood group and date of birth. These are historical UI captures.

15 Individual Report Component B

Dinuwara K.D.S | IT24103033 | Planning and triage orchestration

1. Contribution Statement

As a full-stack contributor to the CareFlow-AI project, I was responsible for key integrations across the backend API, the React Web Admin dashboard, and the Flutter mobile application. My primary contributions focused on ensuring end-to-end functionality, securing deployment pipelines, and enhancing the user experience.

Specifically, I:

- Mobile Application (Flutter): Enhanced the UX of the Patient Profile and Appointments screens, resolved API routing issues by dynamically configuring the mobile app to connect to the deployed Render API, and fixed CI build warnings related to case sensitivity in file structures.
- Web Admin (React/Vite): Developed the logic to sanitize complex input (like Triage Record IDs) in the Prescription Management interface, implemented comprehensive error handling to surface backend validation messages directly in the UI, and configured Vercel routes and ESLint rules to unblock production deployments.
- Backend API (.NET 8): Refactored Triage Approval workflows, resolved authorization logic to allow proper end-to-end demo testing, and implemented PharmacySafetyTests and EF Core in-memory transaction handling to ensure the backend met rigorous unit testing requirements.
- Deployment & CI/CD: Led the final deployment strategy, successfully transitioning the backend to Render and the Web Admin to Vercel, and documented the deployment process in the repository README.md.

2. Evidence of Key Commits & PRs

Commit	Contribution
214eff4	fix: configure mobile services for deployed Render API
a847f48	fix: support React routes on Vercel
3f2769e	chore: relax eslint rules to prevent deployment failures
74c978b	fix: sanitize triage record ID and display backend validation errors in prescription form
e1156d5	fix: resolve triage approval permissions and mobile login profile fetch
546a05f	feat: Enhance patient profile and appointments UX
eea64f4	Fix CS7036 in PharmacySafetyTests and integrate E2E test changes

See GitHub repository for PR #34 and PR #33 which were merged by IT24103033 covering Integration Testing and UI Updates.

15 Individual AI Usage Log

Dinuwara K.D.S | IT24103033

3. Individual AI Usage Log

During this project, I utilized AI (specifically Google Gemini) as an active pair-programming assistant.

Date / Phase	Task / Problem	AI Tool Used	How AI Assisted
Development	Mobile UI & Routing	Gemini	Assisted in refactoring Flutter screens (Patient Profile) and fixing Dart case-sensitivity issues that were breaking the CI pipeline.
Testing	Backend E2E & Unit Tests	Gemini	Suggested fixes for EF Core in-memory database transaction errors and helped resolve CS7036 compilation errors in PharmacySafetyTests.
Debugging	Web Admin API Integration	Gemini	Analyzed silent failures in the Prescription submission form; suggested input sanitization (.replace()) and mapped 400 Bad Request data explicitly to UI error states.
Deployment	CI/CD Pipeline Failures	Gemini	Diagnosed GitHub Actions "red cross" deployment failures. Recommended migrating the backend to Render, configuring Vercel for the frontend, and bypassing overly strict ESLint build blockers.

15 Individual Reflection and Declaration

Dinuwara K.D.S | IT24103033

4. One-Page AI Reflection

Integrating an agentic AI assistant into my software engineering workflow fundamentally changed how I approached problem-solving, particularly during the integration and deployment phases of the CareFlow-AI project.

Effectiveness and Productivity

The AI excelled as a rapid diagnostic tool. For example, when our web application's prescription form was silently failing, the AI quickly analyzed the frontend-backend data contract and identified that the input field was appending unnecessary text (e.g., "Low") to a GUID. Rather than spending hours tracing the network tab and debugging C# controllers, the AI immediately suggested sanitizing the string in React and exposing the backend's 400 Bad Request validation array to the user interface. This turned a frustrating bug into a 5-minute fix. Similarly, the AI proved invaluable for deployment. When GitHub Actions failed due to missing cloud SSH keys and ESLint warnings, the AI correctly contextualized that these were infrastructure and tooling configurations, not core logic flaws. It seamlessly pivoted our strategy to use Render for the backend and Vercel for the frontend, writing the exact `eslint.config.js` and configurations needed to achieve a green build.

Limitations and Challenges

However, the AI is not infallible. A recurring limitation I observed was its tendency to apply overly broad solutions if not given precise context. For instance, when asking the AI to "fix the build," it occasionally attempted to modify core architecture rather than simply adjusting a linter rule. I learned that prompt engineering is critical; treating the AI like a junior developer—providing exact file constraints and clear objectives—yielded much better code. Additionally, the AI lacks intrinsic knowledge of our specific university grading rubrics, meaning I had to manually override its suggestions when it tried to remove "redundant" files (like GitHub Action workflows) that were actually required for our assignment submission.

Impact on Software Engineering

Ultimately, the AI shifted my role from "syntax writer" to "systems architect." Because the AI could quickly generate boilerplate code and fix syntax errors, I was able to spend more cognitive energy on higher-level system design, user experience, and ensuring that the React frontend, .NET backend, and Flutter mobile app all communicated flawlessly. It highlighted that the future of software engineering relies heavily on code review, system orchestration, and critical thinking, rather than just memorizing API documentation.

5. Signed Declaration

I declare that the work presented in this section, as well as the commits attributed to me in the repository, represents my own contribution to the project. Where AI tools were utilized, they were used strictly as an assistant for debugging, scaffolding, and optimizing code, and all AI-generated logic was thoroughly reviewed, tested, and integrated by me.

Signature: _____ (IT24103033)

Date: 06 October 2026

15 Component B Evidence - Interface and Project Board

Dinuwara K.D.S | IT24103033

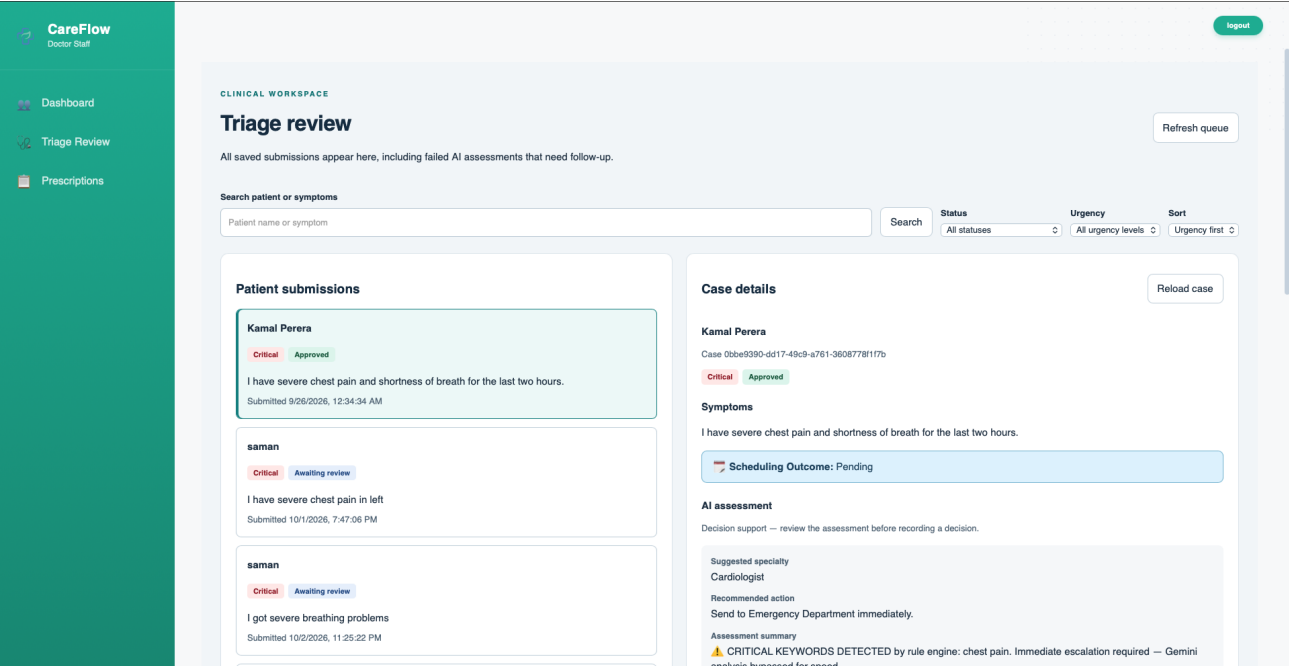


Figure B1. Doctor triage review interface: searchable submissions, urgency and status filters, case details, assessment output and scheduling status. The selected case shows a critical rule-based escalation and a pending scheduling outcome.

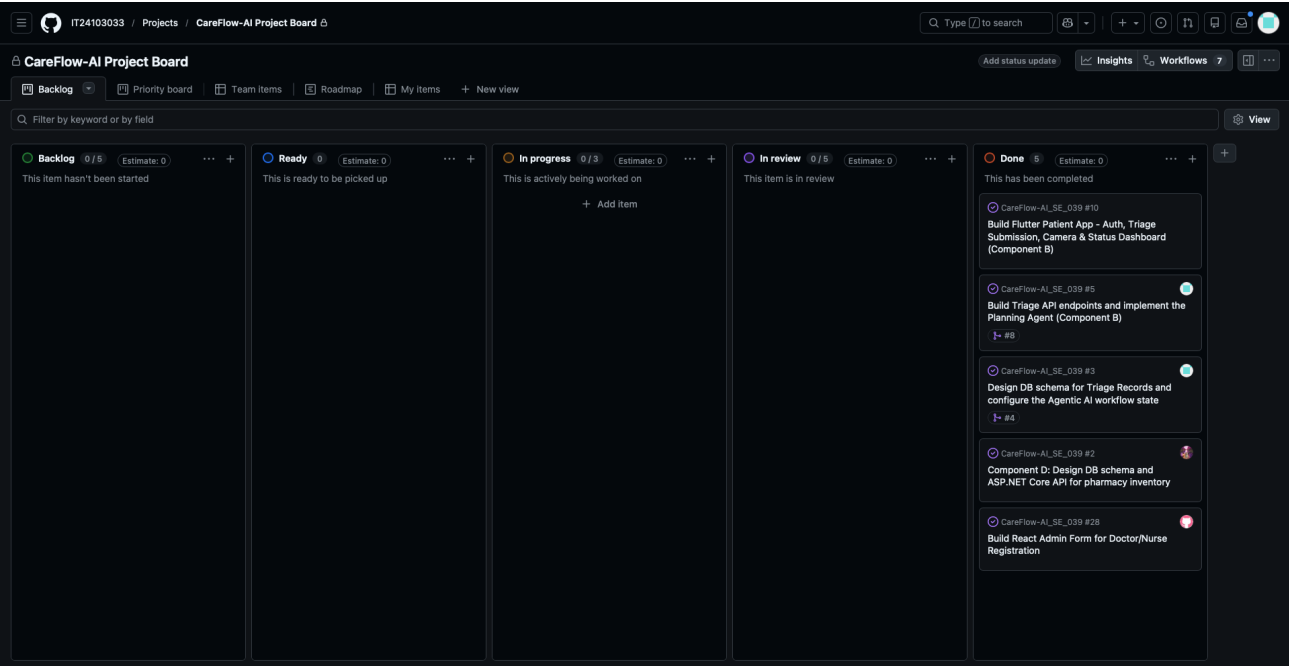


Figure B2. CareFlow-AI GitHub project board. Component B mobile application, triage API/planning agent and triage database/workflow tasks appear in Done. The board also displays tasks belonging to other team members.

16 Individual Report Component C

Pallawala S R | IT24102852 | Appointments and resource scheduling

Individual contribution

I was responsible for appointment scheduling, doctor availability, conflict checking and the Action/Tool Agent. I also contributed to React and Flutter appointment interfaces and the tests and CI work associated with Component C.

Backend, database and agent work

My work covered doctor availability management, available-slot retrieval, appointment date/time validation, tentative booking and doctor approval before confirmation. Doctors, DoctorAvailabilities and Appointments hold the scheduling data. Transaction-based conflict checks were introduced to reduce concurrent double-booking risks.

The Action/Tool Agent uses allow-listed FindAvailableSlots, CheckBookingConflict and CreateTentativeBooking tools. Injected provider abstractions avoid direct calls to protected controller endpoints. AppointmentActionResult and AppointmentActionStatus distinguish successful bookings, unavailable slots, conflicts, invalid requests and provider failures. AppointmentWorkflowRunner connects these contracts to the workflow layer.

Client integration and technical learning

I connected Flutter appointment screens to real doctor, availability and booking APIs and implemented the React appointment page. Challenges included dependency-injection registration, Git conflicts and B/C contract mismatches. I learned to align interfaces and authentication across components and to verify AI suggestions through builds, tests and API checks.

Git evidence

Commit	Contribution
c2c3ed8	Appointment scheduling foundation.
34b4241	Doctor availability APIs.
a7fbcde	Appointment scheduling fixes and integration.

Feature branch: feature/Resource-Scheduling. The supplied document identifies the PR by title; local history also contains merges #9 and #24 from this branch. Shared B/C integration changes must retain their original attribution.

Testing evidence and its limits

Sandathi reports historical local results of 92/92 backend tests and 13/13 Flutter tests, plus a successful Flutter CI check. Her report also records a later B/C compilation mismatch in TriageOrchestrationHelper.cs. These are historical observations, not the current release status. This document update did not rerun the tests.

The consolidated report separately records the inspected baseline results. Its scheduling concurrency test was skipped under EF InMemory; the supplied report does not establish current passing PostgreSQL concurrency coverage.

16 Individual AI Usage Log

Pallawala S R | IT24102852

Task	AI assistance	Reported outcome
Appointment API design	Used AI to understand how to structure doctor availability and appointment scheduling endpoints.	Helped organize the Component C implementation and identify required validation.
Dependency injection	Used AI to understand and review dependency injection and provider/adaptor registration for the appointment tools.	Supported the move away from direct protected-controller HTTP calls.
Action/Tool Agent	Used AI to reason about the allow-listed appointment tools and the agent workflow.	Supported the FindAvailableSlots, CheckBookingConflict and CreateTentativeBooking tool structure.
Structured results	Used AI to review how appointment operations should distinguish success, unavailable slots, conflicts and provider errors.	Resulted in the AppointmentActionResult/AppointmentActionStatus approach.
Concurrency	Used AI to understand concurrent booking risks and transaction isolation.	Supported the serializable transaction approach used for conflict-free booking.
Approval workflow	Used AI to review the requirement that appointments must be approved before confirmation.	Supported the Pending/Approved/Confirmed appointment flow.
Flutter integration	Used AI for step-by-step guidance while connecting appointment screens to real availability and booking APIs.	Helped replace temporary/sample appointment data with API-driven behaviour.
Git/merge conflicts	Used AI to understand Git conflict markers and resolve the Component C Program.cs registration conflict.	Helped retain the provider-based Component C registrations.
CI debugging	Used AI to interpret GitHub Actions build/test logs.	Helped identify the difference between Component C implementation issues and shared B/C integration issues.
Documentation	Used AI to organize contribution evidence, testing evidence and the individual AI reflection.	Produced a structured submission section for final review.

Student-reported review approach: AI suggestions were reviewed and adapted to the existing architecture. Her reflection describes checking project structure, authentication and interfaces, and verification through builds, automated tests, API testing and CI.

16 Individual Reflection and Declaration

Pallawala S R | IT24102852

Individual AI reflection

During the development of Component C, I used AI as a supporting tool to understand technical concepts, design parts of the appointment workflow, debug errors, and improve my testing approach. I used AI suggestions as guidance and adapted them to the existing CareFlow-AI_SE_039 codebase.

AI was particularly helpful when working with dependency injection, provider abstractions, and the Action/Tool Agent. It helped me understand why the appointment tools should use injected application services instead of directly calling protected controller endpoints. AI also helped me design structured appointment results so that situations such as unavailable slots, booking conflicts, invalid requests, and provider errors could be handled separately.

AI also helped me understand concurrency and approval control. I learned why checking availability and creating a booking as separate operations could result in conflicting concurrent bookings. This supported the use of a serializable database transaction. I also used AI to understand the human-in-the-loop approval process, where an appointment remains tentative until it is approved by a doctor before confirmation.

During Flutter integration, Git conflict resolution, and CI troubleshooting, AI helped me identify problems and understand possible solutions. However, I learned that AI-generated suggestions should not be accepted without verification. I had to check the suggestions against the existing project structure, authentication requirements, interfaces, and other team members' work.

Overall, AI improved my productivity and helped me understand difficult technical problems more quickly. However, testing and verification remained my responsibility. I validated my implementation through builds, automated tests, API testing, and CI results. My main lesson is that AI is most useful as a development assistant when its suggestions are understood, adapted, and tested rather than accepted automatically.

Individual declaration

I declare that the work and contribution described in this individual section accurately represent my contribution to the CareFlow-AI_SE_039 project. I have identified the use of AI assistance honestly and have not intentionally presented another member's contribution as my own. I understand that the submitted evidence should correspond to the project repository, commits, pull requests and test results.

Student: Pallawala S R | IT24102852 Date: 05/10/2026

Signature:



16 Component C Evidence Git and Review

Screenshots supplied in the individual report. They record development history and do not establish the final release status.

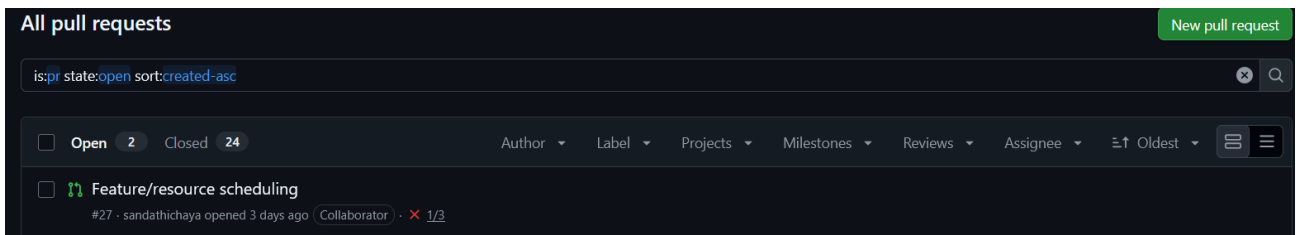


Figure C1. Resource-scheduling PR #27 listed as open, with a failed check indicator at the time of capture.

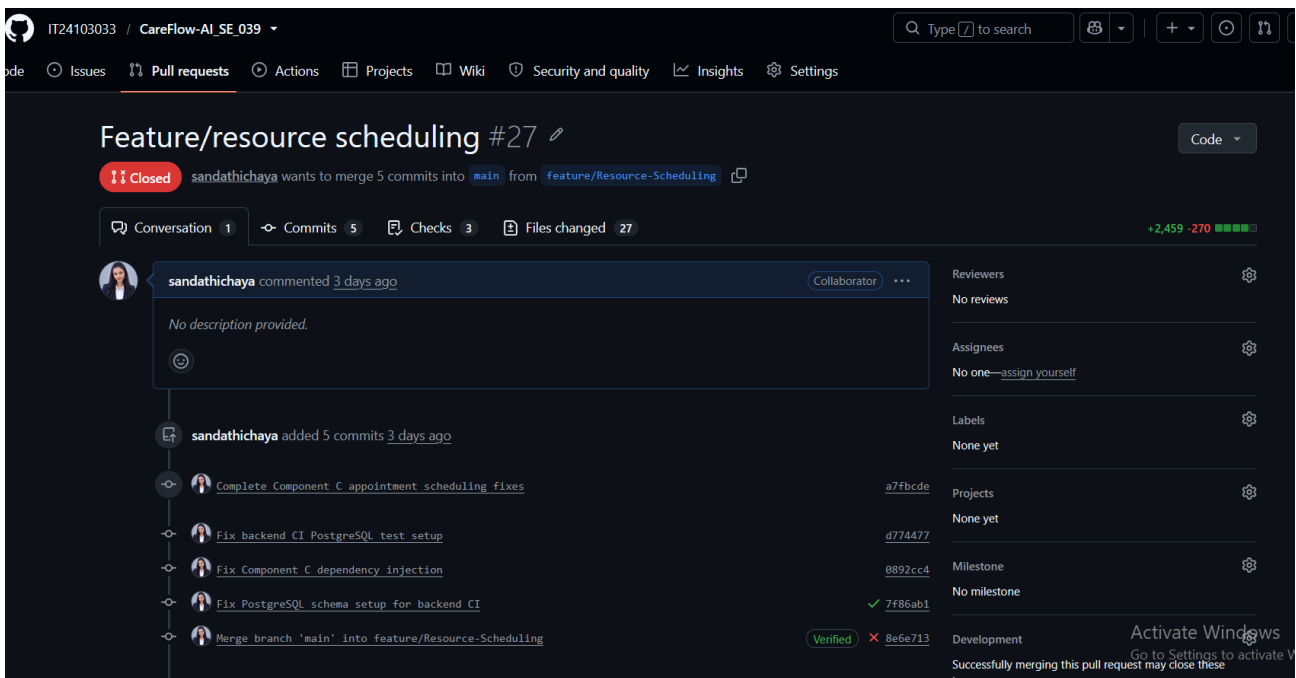


Figure C2. A later view shows PR #27 closed and lists scheduling, dependency-injection and PostgreSQL CI fixes. Closed is not evidence of a direct merge; integration evidence is recorded separately.

16 Component C Evidence Cloud Database

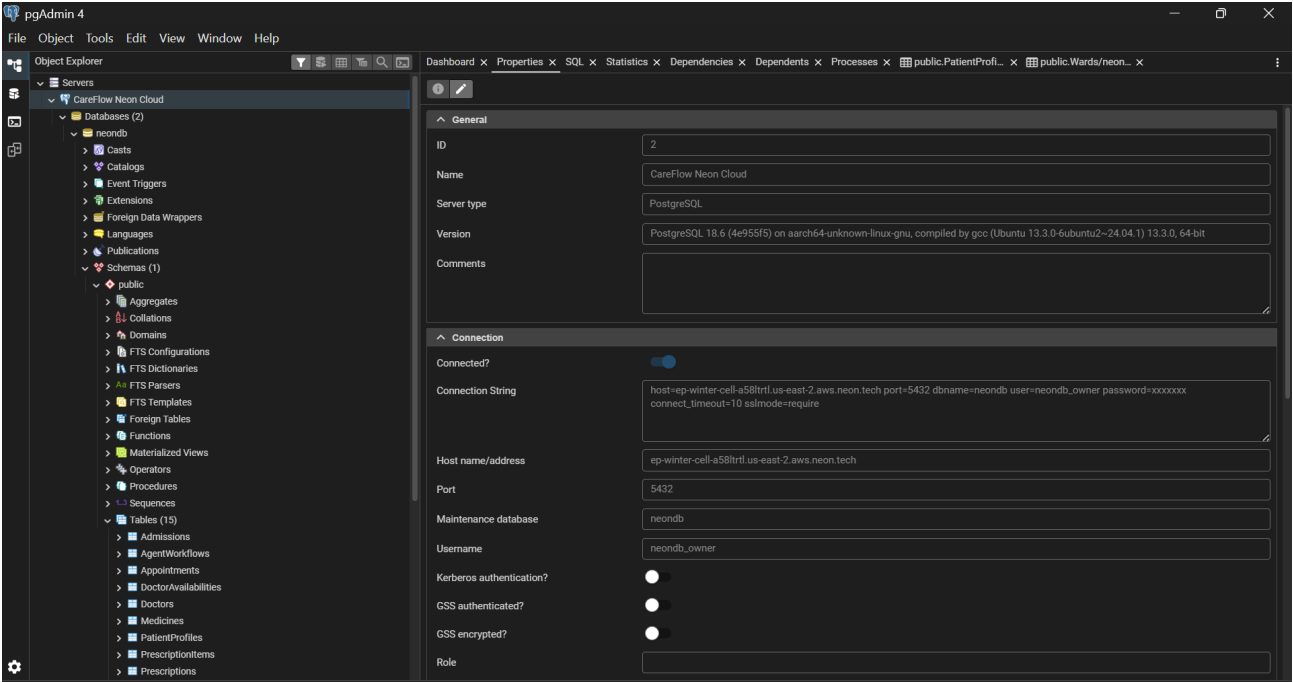
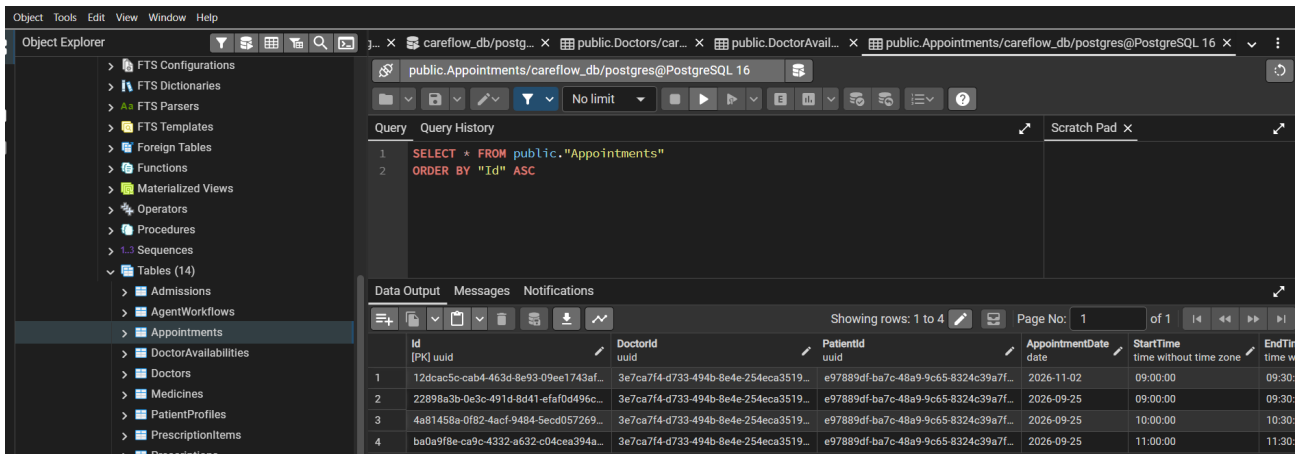


Figure C3. Supplied pgAdmin view of a connected Neon PostgreSQL database and its table list. The password is masked in the source image. This view alone does not demonstrate synchronization or an end-to-end workflow.

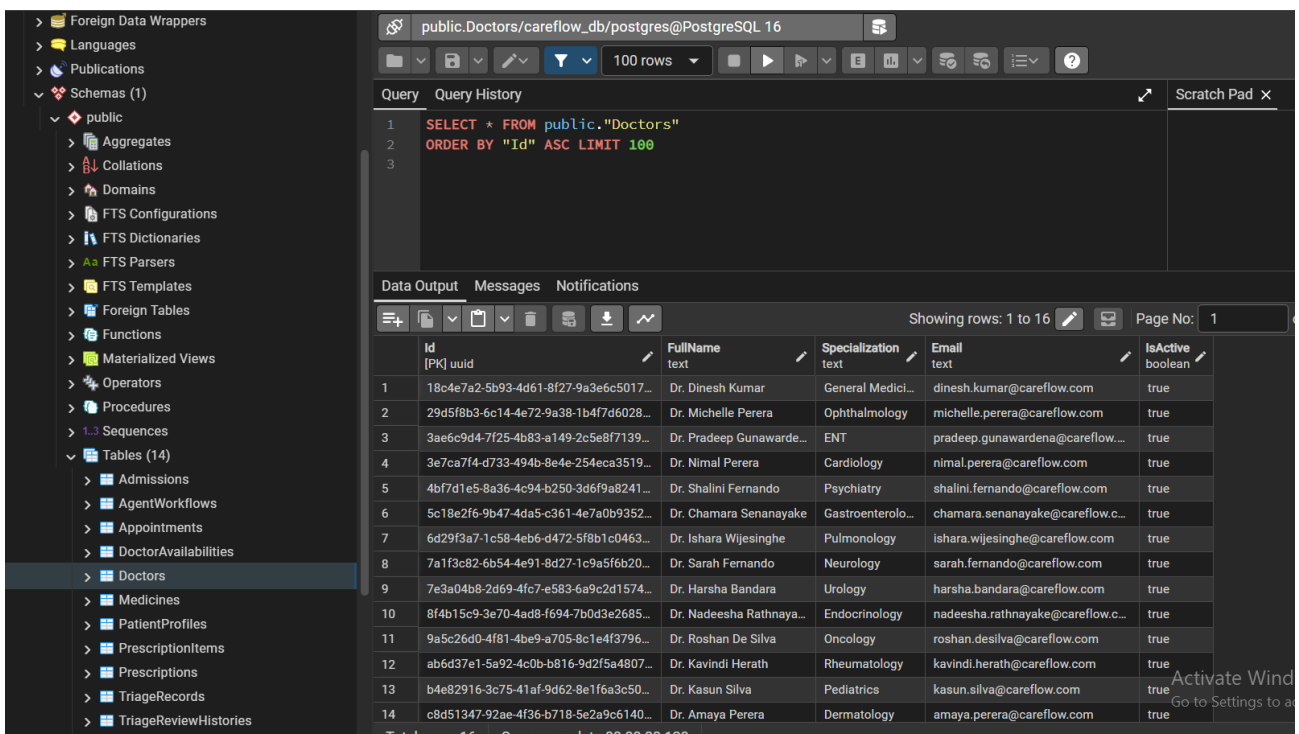
16 Component C Evidence Persisted Records



The screenshot shows a PostgreSQL query interface. The query is: `SELECT * FROM public."Appointments" ORDER BY "Id" ASC`. The results are displayed in a table with columns: Id, DoctorId, PatientId, AppointmentDate, StartTime, and EndTime. The results show four rows of appointment data.

Id	DoctorId	PatientId	AppointmentDate	StartTime	EndTime
1	12dca5c-cab4-463d-8e93-09ee1743af...	3e7ca7f4-d733-494b-8e4e-254eca3519...	2026-11-02	09:00:00	09:30:00
2	22898a3b-0e3c-491d-8d41-efaf0d496c...	3e7ca7f4-d733-494b-8e4e-254eca3519...	2026-09-25	09:00:00	09:30:00
3	4a81458a-0f82-4acf-9484-5ecd057269...	3e7ca7f4-d733-494b-8e4e-254eca3519...	2026-09-25	10:00:00	10:30:00
4	ba0a9f8e-ca9c-4332-e632-c04cea394a...	3e7ca7f4-d733-494b-8e4e-254eca3519...	2026-09-25	11:00:00	11:30:00

Figure C4. Local PostgreSQL Appointments query showing four stored rows with doctor, patient and scheduling fields.



The screenshot shows a PostgreSQL query interface. The query is: `SELECT * FROM public."Doctors" ORDER BY "Id" ASC LIMIT 100`. The results are displayed in a table with columns: Id, FullName, Specialization, Email, and IsActive. The results show 14 rows of doctor data.

Id	FullName	Specialization	Email	IsActive
1	Dr. Dinesh Kumar	General Medici...	dinesh.kumar@careflow.com	true
2	Dr. Michelle Perera	Ophthalmology	michelle.perera@careflow.com	true
3	Dr. Pradeep Gunawardena	ENT	pradeep.gunawardena@careflow.com	true
4	Dr. Nimal Perera	Cardiology	nimal.perera@careflow.com	true
5	Dr. Shalini Fernando	Psychiatry	shalini.fernando@careflow.com	true
6	Dr. Chamara Senanayake	Gastroenterolo...	chamara.senanayake@careflow.c...	true
7	Dr. Ishara Wijesinghe	Pulmonology	ishara.wijesinghe@careflow.com	true
8	Dr. Sarah Fernando	Neurology	sarah.fernando@careflow.com	true
9	Dr. Harsha Bandara	Urology	harsha.bandara@careflow.com	true
10	Dr. Nadeesha Rathnayake	Endocrinology	nadeesha.rathnayake@careflow.c...	true
11	Dr. Roshan De Silva	Oncology	roshan.desilva@careflow.com	true
12	Dr. Kavindi Herath	Rheumatology	kavindi.herath@careflow.com	true
13	Dr. Kasun Silva	Pediatrics	kasun.silva@careflow.com	true
14	Dr. Amaya Perera	Dermatology	amaya.perera@careflow.com	true

Figure C5. Local PostgreSQL Doctors query showing doctor names, specializations and active status. This is a different database view from the cloud connection in Figure C3.

16 Component C Evidence Appointment Interface

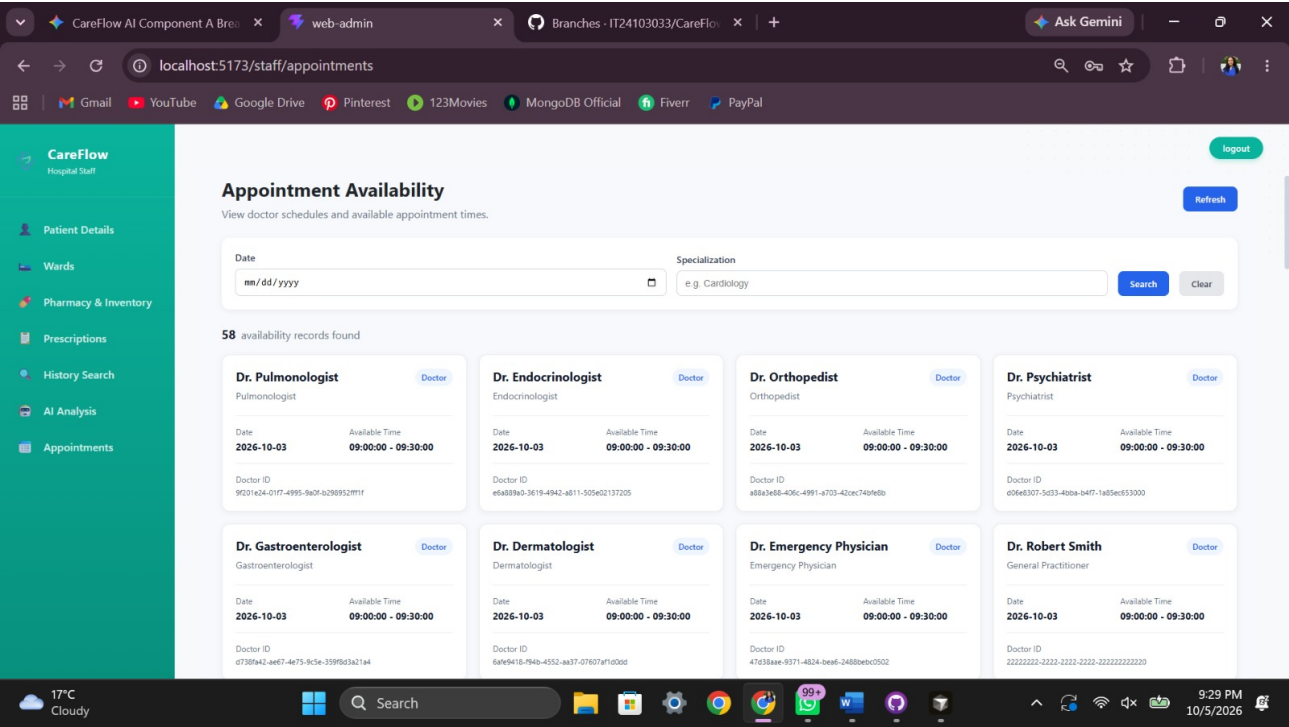


Figure C6. Local React staff appointment-availability page showing 58 availability records and date/specialization filters.

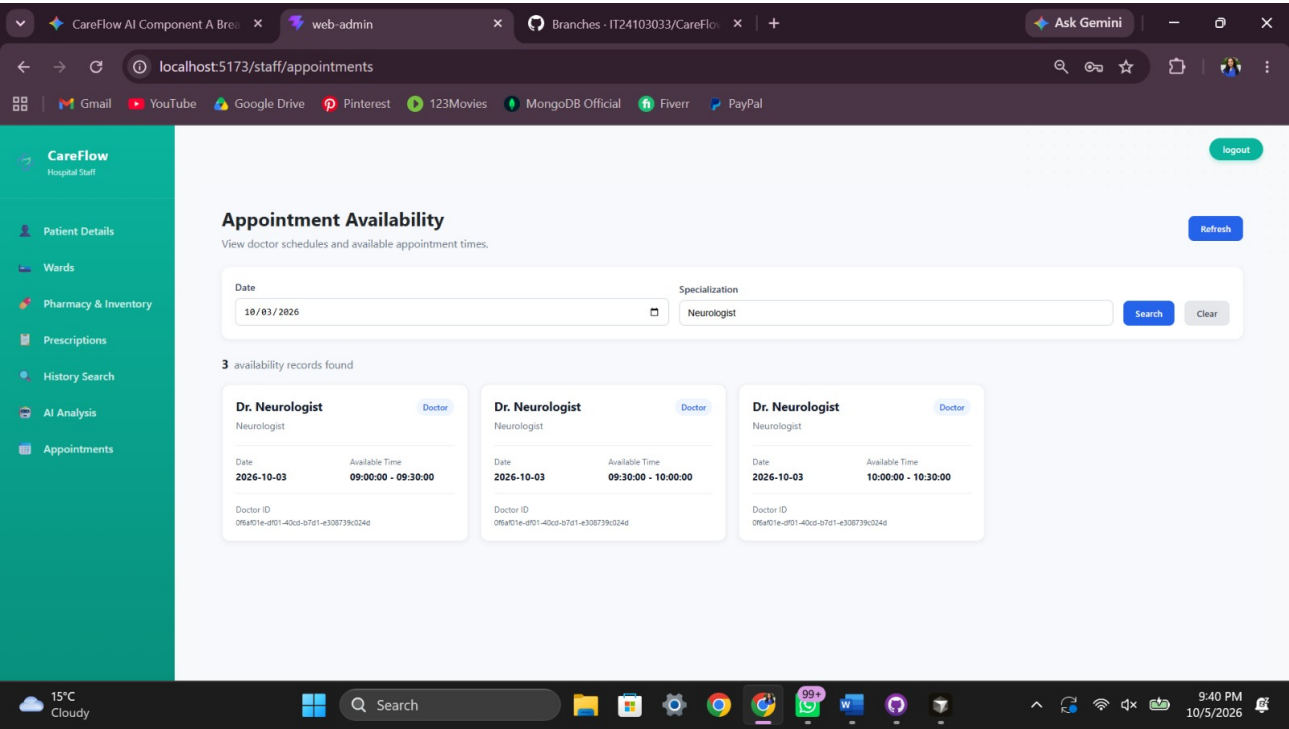


Figure C7. Filtered availability for a neurologist on 3 October 2026, showing three time intervals. These UI captures show availability browsing, not successful booking or doctor approval.

17 Individual Report Component D

Dulshan P A | IT24102599 | Pharmacy inventory, e-prescriptions and safety verification

Individual contribution

My responsibility was Component D: medicine inventory, e-prescriptions, safety validation, dispensing and notifications. I implemented inventory CRUD, stock and expiry tracking, low-stock handling, prescription creation and approval, and the pharmacy dispensing workflow. I also worked on related React and Flutter interfaces, migrations, automated tests and integration fixes.

Backend, agent and patient features

The component uses Medicines, Prescriptions and PrescriptionItems, with MedicinesController, PrescriptionsController, PharmacyAiService and NotificationService. My safety work covered unsafe approval, approving-doctor identity and active status, patient/triage consistency, duplicate medicines, drug interactions, emergency rules and duplicate stock deductions.

I added SMS/email integration, patient contact fields, notification status, timestamps, failure details and retry tracking. Patient features included QR prescription pickup and medication reminders. I also contributed Docker and GitHub Actions configuration. Provider delivery, device reminders and deployed operation require separate runtime evidence.

Traceable contribution evidence

Commit / PR	Contribution
6eadd16 / #6	Inventory, prescriptions, interfaces and migrations.
89ecf78 / #12	SMS/email notification integration.
88b01eb / #16	Safety, authorization, ownership and stock concurrency.
db70e2d, 68ffbe2 / #20	QR pickup, reminders, Docker and CI fixes.
69c0982, 725e22c / #26	Notification tracking/retry, contact fields and migration integration.

Branches: feature/inventory, InventoryNew, fix/dulshan and bugfix/dulshan. Local Git history confirms the listed PR merges, including #26 (27182a5). Shared integration fixes, including eea64f4, are not claimed as sole authorship.

Testing evidence

Dulshan reports 24 passed, 0 failed and 0 skipped in PharmacySafetyTests. The current file contains 24 Fact tests covering approval safety, identity, ownership, duplicate dispensing/medicines, emergency rules, notification failures, contact data, retry, access control and missing configuration.

Command: dotnet test backend-api.Tests/CareFlowAI.API.Tests.csproj --filter "FullyQualifiedName~PharmacySafetyTests"

17 Personal Evidence and Reflection

Dulshan P A | IT24102599

Individual AI usage log

Date	Task assisted by AI	Student review and verification
2026-09-21	Inventory and e-prescription implementation	Reviewed generated code, tested and changed it before committing.
2026-09-25	SMS/email notification integration	Reviewed, configured the service and tested the workflow.
2026-09-28	Safety, authorization, concurrency and tests	Reviewed changes and ran tests before committing.
2026-10-02	QR pickup, reminders, Docker and CI	Debugged, changed the implementation and verified changes.
2026-10-02	Notification retry and migration conflicts	Reviewed migrations, fixed conflicts and tested the system.

Individual reflection

I used Antigravity with Gemini during the development of Component D to help with implementation, debugging and testing.

It was useful when working with ASP.NET Core, database migrations, JWT authentication, Flutter notifications, Docker and GitHub Actions.

I did not use the generated code without checking it. I reviewed the code, made changes where needed, ran the application and tested the features before committing them.

For safety-related features, I paid extra attention to the generated code because incorrect validation could affect prescription approval and dispensing.

AI also helped me find and fix development and CI/CD issues more quickly.

Through this work, I improved my understanding of API development, database migrations, authentication, testing, notification services and deployment.

Overall, I used AI as a development assistant. I was still responsible for reviewing, testing and deciding which changes should be included in the project.

Individual declaration

I declare that the information in this document accurately represents my individual contribution to Component D of the CareFlow AI project. I confirm that the Git branches, commits, pull requests and test results mentioned in this document are related to my contribution.

Signature: _____ Date: _____

18 Consolidated AI Usage Declaration

We acknowledge using AI assistance where permitted for development, debugging, documentation and verification. Each member is responsible for the accuracy and understanding of their submitted work and for an individual usage log. We distinguish development assistants from the application's own agent subsystem. We will not use external AI assistance during the final demonstration or viva.

Known use in preparing this report

Development assistance included Google Gemini, Antigravity and Codex. Gemini and Antigravity supported implementation and debugging as described in the individual logs. Codex supported source and Git inspection, test execution, deployment configuration, documentation, diagrams and PDF preparation. Individual contribution statements and reflections were provided by the members. Each member remains responsible for reviewing, understanding and explaining their submitted work.

Member	Signature	Date
Bandara MMSD / IT24103089		
Dinuwara K.D.S / IT24103033		
Pallawala S R / IT24102852		
Dulshan P A / IT24102599		

19 References and Evidence Index

R1

SE3090 Assignment 1 Specification with Marking Scheme, 2026, 17 pages. Sections 5 to 15 define technical and submission requirements; sections 18 to 20 define disclosure and final checks.

R2

CareFlow source repository: https://github.com/IT24103033/CareFlow-AI_SE_039 . Inspected baseline db3a50f with local modifications on 5 October 2026.

E1

backend-api.Tests: current run recorded 101 passed, 1 skipped; suite includes scheduling, planning, review, authentication and pharmacy tests.

E2

web-admin/tests/triage.test.js: 10 passing contract tests; npm run build completed.

E3

Flutter verification: 12 earlier triage tests passed; subsequent flutter analyze --no-pub completed with no issues following deployment configuration changes.

E4

backend-api/Data/ApplicationDbContext.cs and Migrations: entity relationships, tokens, seed configuration and migration history.

E5

backend-api/Services/Planning and ai-orchestrator: plan schema, tool permissions and agent contracts.

E6

Git commits db3a50f, 0df54ce, eea64f4, 9a2a30d, f3c14b8 and scheduling/pharmacy component commits cited in individual sections.

E8

Member documents: Individual Contribution Report.docx (A), ComponentD.docx (D) and SE3100_Individual_Section_Component-c.docx (C). Contributions, logs and original reflections; A/C include signatures. B supplied his individual section directly on 6 October 2026. Individual AI logs are included in the member sections.

E7

Historical read-only database observations from 4 and 5 October, summarized without personal symptom text. Earlier-build evidence only; not a substitute for the final release demonstration.

19 Submission Access and Evaluation Guide

Deliverable	Location
Source repository	https://github.com/IT24103033/CareFlow-AI_SE_039
Web application	https://care-flow-ai-se-039.vercel.app
Backend API	https://careflow-ai-se-039.onrender.com
Android APK	Download Android APK from Drive
Demonstration video	Open demonstration video folder

Evaluator access

Web accounts: admin@careflow.ai, doctor@careflow.ai and staff@careflow.ai. The password for each demonstration account is password. Patient users can open the mobile app, select Create Account, register with test details and sign in.

System walkthrough

Start with a patient symptom submission in the mobile application. Inspect the assessment and available appointment options, then use the doctor portal to review the case. Follow the resulting appointment and prescription states through the corresponding web and mobile screens. The recorded demonstration accompanies the implementation evidence in this report.

Operational limitations

The Render free instance can experience cold-start delays. Model responses depend on provider availability and quota. Assessment failures remain explicit and do not represent a completed clinical recommendation. Email/SMS delivery and concurrent PostgreSQL race behavior are not established by the recorded unit tests. CareFlow AI is an academic prototype with human review of clinical workflow decisions.